Array Class 02:

–....>>Is arry a object ?

```
int[] arr = {1, 2, 3, 4};
System.out.println(arr); // Prints something like '[I@1b6d3586'
```

Output might look like:

```
text
```

```
[I@1b6d3586
```

`[I` means an array of int type.

`1b6d3586` is the hash code in hexadecimal, not the memory address.

====

```
double[] darr = {1.1, 2.2, 3.3, 4.4};
System.out.println(darr); // Output: [D@36baf30c

float[] farr = {1.1f, 2.2f, 3.3f, 4.4f};
System.out.println(farr); // Output: [F@3feba861

String[] sarr = {"A", "B", "C", "D"};
System.out.println(sarr); // Output: [Ljava.lang.String;@5fa07e12
```

Explanation of Output

- [D means array of double
- [F means array of float
- [Ljava.lang.String; means array of String objects
- The part after `@` is the object's hash code in hexadecimal (not its memory location)

While loop

```
class ArrayIteration2{
    public static void main(String[] args) {
        int [] array2 = {1,2,3,4};
        int count = 0;
        while (count < array2.length) {
            System.out.println(array2[count]);
            count++;
        }
    }
}
```

18. Arrays.toString() use case

Why Use Arrays.toString()?

- Printing an array directly (e.g., `System.out.println(arr);`) displays its type and hash code, not its elements.
- Arrays.toString() creates a string representation of the array, formatting its elements within square brackets and separated by commas.
- It works for all primitive arrays and arrays of objects (for objects: `toString()` must be overridden for meaningful output)

Array Operations

- **String representation of an array:**
  - We cannot print array elements directly in Java as it shows the memory location of that array object.
  - **Arrays.toString()** method is used to represent an array.
  - The array elements are separated by comma and enclosed by square brackets [ ].

```
import java.util.Arrays;
String [] courses = { "CSE110", "PHY111", "MAT110", "ENG101"};
System.out.println(courses);
System.out.println(Arrays.toString(courses));
```

```
Welcome to DrJava. Working directory is C:\Users\Desktop
[Ljava.lang.String;@379619aa
[CSE110, PHY111, MAT110, ENG101]
```

Basic Example

java

```java
import java.util.Arrays;

int[] n = {1, 2, 3, 4};
System.out.println(Arrays.toString(n));
// Output: [1, 2, 3, 4]
```

Multiple Data Types Example

java

```java
import java.util.Arrays;

double[] darr = {1.1, 2.2, 3.3, 4.4};
System.out.println(Arrays.toString(darr));
// Output: [1.1, 2.2, 3.3, 4.4]

String[] sarr = {"A", "B", "C"};
System.out.println(Arrays.toString(sarr));
```

```
// Output: [A, B, C]
```

==========

Here are examples of array operations in Java without using functions or methods—all code is inside the `main` method for simplicity:

java

```java
public class ArrayOperations {
    public static void main(String[] args) {

        // 1. Length
        int[] arr = {10, 20, 30, 40, 50};
        System.out.println("Length: " + arr.length);

        // 2. Traverse
        System.out.print("Traverse: ");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();

        // 3. Insertion (insert 99 at index 2)
        int index = 2, value = 99;
        int[] insertArr = new int[arr.length + 1];
        for (int i = 0; i < index; i++) {
            insertArr[i] = arr[i];
        }
        insertArr[index] = value;
        for (int i = index; i < arr.length; i++) {
            insertArr[i + 1] = arr[i];
        }
        System.out.print("After insertion: ");
        for (int i = 0; i < insertArr.length; i++) {
            System.out.print(insertArr[i] + " ");
        }
        System.out.println();
```

```
        // 4. Deletion (delete element at index 1)
        int delIndex = 1;
        int[] delArr = new int[arr.length - 1];
        for (int i = 0; i < delIndex; i++) {
            delArr[i] = arr[i];
        }
        for (int i = delIndex + 1; i < arr.length; i++) {
            delArr[i - 1] = arr[i];
        }
        System.out.print("After deletion: ");
        for (int i = 0; i < delArr.length; i++) {
            System.out.print(delArr[i] + " ");
        }
        System.out.println();

        // 5. Update (change value at index 3 to 333)
        arr[10] = 333;
        System.out.print("After update: ");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();

        // 6. Copying an Array
        int[] copyArr = new int[arr.length];
        for (int i = 0; i < arr.length; i++) {
            copyArr[i] = arr[i];
        }
        System.out.print("Copied array: ");
        for (int i = 0; i < copyArr.length; i++) {
            System.out.print(copyArr[i] + " ");
        }
        System.out.println();
    }
}
```

This code covers:

- Getting the length
- Traversing the array
- Insertion by creating a bigger array
- Deletion by creating a smaller array
- Updating elements directly
- Copying with a loop.

==========

find the maximum element in an array:

java

```java
public class MaxElement {
    public static void main(String[] args) {
        int[] arr = {24, 17, 42, 3, 89, 5};

        int max = arr[0]; // Assume first element is max
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i]; // Update max if current element is
greater
            }
        }

        System.out.println("Maximum element: " + max);
    }
}
```

Output:

text

```
Maximum element: 89
```

This code loops through each element, comparing and updating the `max` variable when a larger value is found. The final result is the largest value in the array.

==========

Here is a simple Java code example to find the smallest element in an array:

java

```java
public class MinElement {
    public static void main(String[] args) {
        int[] arr = {24, 17, 42, 3, 89, 5};

        int min = arr[0]; // Assume first element is the minimum
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) {
                min = arr[i]; // Update min if current element is
smaller
            }
        }

        System.out.println("Smallest element: " + min);
    }
}
```

Output:

text

```
Smallest element: 3
```

This code checks each element, updating the `min` value whenever a smaller number is found, providing the minimum element in the array.

Assignment 1 👍

```java
import java.util.Scanner;

public class LabArrayTask {
```

```java
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Part a: Take N and the array elements from user
        System.out.print("Enter the number of elements (N): ");
        int N = sc.nextInt();
        int[] arr = new int[N];

        for (int i = 0; i < N; i++) {
            System.out.print("Enter a number: ");
            arr[i] = sc.nextInt();
        }

        // Print the array elements with their indices
        System.out.println("The elements of the array are:");
        for (int i = 0; i < arr.length; i++) {
            System.out.println(i + ": " + arr[i]);
        }

        // Part b: Take another input, resize array, add new
element, and print
        System.out.print("Enter another number: ");
        int newElement = sc.nextInt();
        int[] newArr = new int[arr.length + 1];

        for (int i = 0; i < arr.length; i++) {
            newArr[i] = arr[i];
        }
        newArr[newArr.length - 1] = newElement;

        System.out.println("After resizing the array:");
        for (int i = 0; i < newArr.length; i++) {
            System.out.print(newArr[i] + " ");
        }
        System.out.println();
```

```
    }
}
```

This code matches your required input/output style:

- The user is prompted for each number.
- Elements are printed with their indices.
- After resizing and adding a number, the final array is printed in a single line, space-separated.

//

Assignment 02:

Here is a Java program that:

- Takes the length and elements of an array from user input.
- Reverses the array (A) using a new array (out-of-place), prints it.
- Reverses the original array in place (B), prints it.

```
java

import java.util.Scanner;

public class ReverseArrayLab {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // User input for array length
        System.out.print("Enter the length of the array: ");
        int N = sc.nextInt();
        int[] arr = new int[N];

        // User input for array elements
        for (int i = 0; i < N; i++) {
            System.out.print("Enter a number: ");
            arr[i] = sc.nextInt();
        }
```

```
        // A. Reverse using a new array (out-of-place)
        int[] reversed = new int[N];
        for (int i = 0; i < N; i++) {
            reversed[i] = arr[N - 1 - i];
        }
        System.out.print("Reversed using a new array: ");
        for (int i = 0; i < N; i++) {
            System.out.print(reversed[i] + " ");
        }
        System.out.println();

        // B. Reverse the original array in place
        for (int i = 0; i < N / 2; i++) {
            int temp = arr[i];
            arr[i] = arr[N - 1 - i];
            arr[N - 1 - i] = temp;
        }
        System.out.print("Reversed the original array: ");
        for (int i = 0; i < N; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();

        sc.close();
    }
}
```

- The first reversal creates a new array and copies the elements in the opposite order.
- The second reversal swaps elements from the ends towards the center, changing the original array itself.

// Assignment 03

```java
public class ReplaceDuplicatesWithZero {
    public static void main(String[] args) {
        int arr[] = {9, -5, 7, 9, -5, 5, 7};

        // Print the original array
        System.out.print("Before removing duplicates: ");
        for (int num : arr) {
            System.out.print(num + " ");
        }
        System.out.println();

        // Replace all duplicates with 0
        for (int i = 0; i < arr.length; i++) {
            for (int j = i+1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    arr[j] = 0;
                }
            }
        }

        // Print the updated array
        System.out.print("After replacing duplicates with 0: ");
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }
}
```

Output:

```
text
```

```
Before removing duplicates: 9 -5 7 9 -5 5 7
After replacing duplicates with 0: 9 -5 7 0 0 5 0
```

/// assignment 04

```java
public class UniqueElementsArray {
    public static void main(String[] args) {
        int arr[] = {23, 100, 23, 56, 100};
        int n = arr.length;

        System.out.print("Input array: ");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();

        // Array to store unique elements, max size n
        int unique[] = new int[n];
        int uniqueCount = 0;

        // Check each array element for being a duplicate
        for (int i = 0; i < n; i++) {
            boolean isDuplicate = false;
            for (int j = 0; j < uniqueCount; j++) {
                if (arr[i] == unique[j]) {
                    isDuplicate = true;
                    break;
                }
            }
            if (!isDuplicate) {
                unique[uniqueCount++] = arr[i];
            }
        }

        System.out.print("New array: ");
        for (int i = 0; i < uniqueCount; i++) {
            System.out.print(unique[i] + " ");
        }
        System.out.println();
```

```
    }
}
```

Sample Output:

```
text
```

```
Input array: 23 100 23 56 100
New array: 23 100 56
```